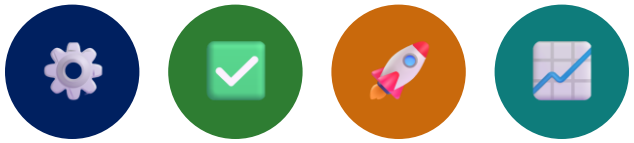
"Automate. Secure. Deliver. Repeat."

MODULE 43

GitHub Actions and CI/CD Integration

Learn how to automate your software delivery process using GitHub Actions for continuous integration and continuous delivery.



TEKsystems Global Services

WORKFORCE DEVELOPMENT

TEKsystems Global Services

WORKFORCE DEVELOPMENT

MODULE 43 OVERVIEW

Automate your software delivery process using GitHub Actions for continuous integration and continuous delivery -- build, test, scan, package, and deploy the CRM with confidence.

This module builds the GitHub Actions CI/CD pipeline that verifies, scans, and packages the Customer Management Platform CRM before Lab 44 promotes the same artifact through staging and production.

DURATION & LAB

1 Hour Theory

CI/CD concepts, GitHub Actions workflows, pipeline architecture, quality gates, and deployment automation.

2 Hours Lab

lab43-crm: verify job, package-once checksum, quality gates, secured variables, forced failure + restore.

Scenario

Northstar Customer Management Platform -- leadership freezes the delivery gate for PRs, main, and tags.

MODULE ARC

Understand CI/CD

Why CI/CD matters and how GitHub Actions automates the software delivery lifecycle.

Build the Pipeline

Configure workflows: build, test, scan, package, and deploy with quality and approval gates.

Ship the Lab

Wire lab43-crm's real GitHub Actions workflow with immutable artifacts and secured secrets.

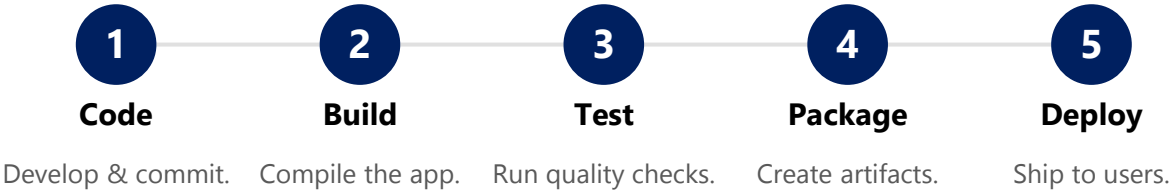
KEY TAKEAWAY Total time: 3 hours (1 hour theory + 2 hours hands-on lab). By the end, you will design and operate a reliable, secure, automated GitHub Actions CI/CD pipeline.

WHY CI/CD MATTERS

CI/CD transforms the way modern teams build, test, and deliver software -- enabling speed, quality, and reliability at scale.

- **Deliver Faster --** automate build, test, and deployment to release features quickly and confidently.
- **Improve Quality --** continuous testing and code analysis catch issues early and improve reliability.
- **Enhance Collaboration --** standardized workflows and automation keep teams aligned and moving forward.
- **Increase Efficiency --** reduce manual tasks and context switching, freeing time for innovation.
- **Ensure Consistency --** automated pipelines deliver the same reliable process across every environment.

CI/CD PIPELINE AT A GLANCE



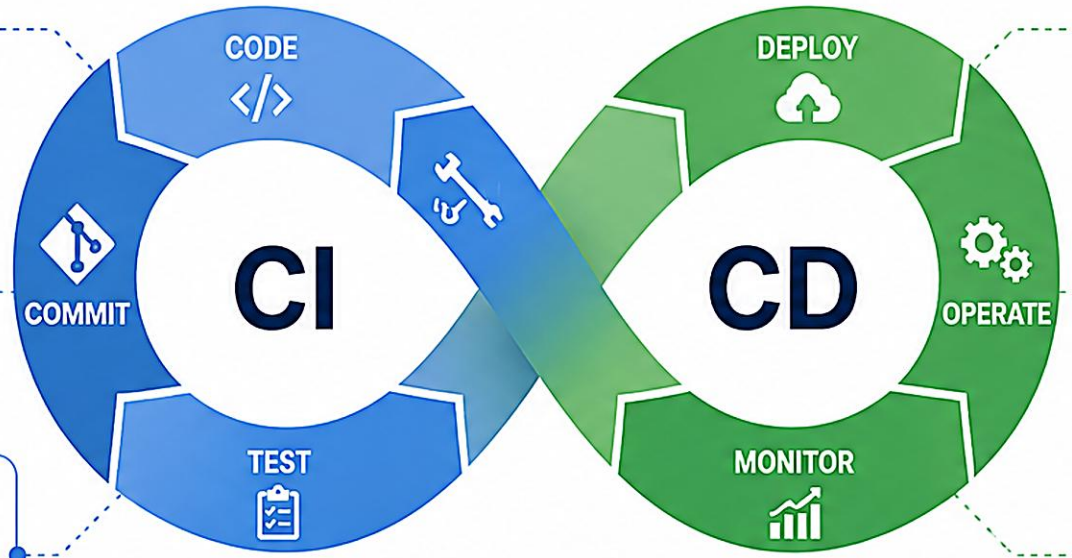
OUTCOMES OF A HEALTHY CI/CD PRACTICE

Faster Time to Market | **Higher Reliability** | **Lower Risk & Cost** | **Happier Customers** | **Business Growth**

KEY TAKEAWAY CI/CD is not just a toolset -- it's a culture of continuous improvement and customer value.

Why CI/CD Matters

CI/CD enables faster delivery, higher quality, and happier teams



Faster Delivery
Automate the pipeline to deliver features faster and more frequently.

Higher Quality
Automated testing and quality checks catch issues early and reduce defects.

Improved Collaboration
Developers, testers, and operations work together with shared visibility and feedback.

Greater Reliability
Consistent, repeatable releases reduce risks and improve system stability.

Reduced Costs
Less manual work, fewer failures, and lower rework result in cost savings.

Business Agility
Respond quickly to market changes and customer needs with confidence.

Business Impact

Faster Time to Market
Deliver value to customers sooner.

Better Customer Satisfaction
Reliable releases and fewer issues improve trust.

Lower Risk
Early detection and automated rollbacks reduce failures.

Cost Efficiency
Automated processes and efficient resource usage.

Developer Productivity
Less manual effort, more time to build and innovate.

Data-Driven Decisions
Metrics and monitoring drive continuous improvement.

MODULE LEARNING OBJECTIVES

By the end of this module, you will be able to build modern, automated CI/CD pipelines that deliver high-quality software faster.

- **1. Understand CI/CD Concepts --** explain the principles of Continuous Integration, Continuous Delivery, and Continuous Deployment and their benefits.
- **2. Work with GitHub and Bitbucket --** manage source code, branches, pull requests, and reviews.
- **3. Use GitHub Actions --** create and configure GitHub Actions workflows to automate builds, tests, and deployments.
- **4. Automate Build and Test --** implement automated builds, run unit tests, perform security scans, and package applications.
- **5. Implement Deployment Automation --** deploy applications to staging and production using automated workflows and approval gates.
- **6. Ensure Code Quality --** apply code quality metrics and Code Insights to maintain high standards and improve code health.
- **7. Apply CI/CD Best Practices --** follow industry best practices to build secure, efficient, and reliable CI/CD pipelines.

KEY TAKEAWAY These objectives will help you build modern, automated CI/CD pipelines that deliver high-quality software faster -- for the Northstar CRM and beyond.

WHAT IS CI/CD?

CI/CD automates code integration, testing, and delivery so high-quality software can be released faster, safer, and more frequently.

CONTINUOUS INTEGRATION (CI)

- Developers frequently integrate code into a shared repository.
- Every change is automatically built and tested.
- Detects integration issues early; keeps the codebase stable.

CONTINUOUS DELIVERY (CD)

- Code that passes all tests is automatically prepared for release.
- Always have a deployable artifact, ready for staging.
- Automated validation gives faster, more reliable releases.

CONTINUOUS DEPLOYMENT (CD)

- Every change that passes the pipeline deploys to production.
- No manual intervention -- fully automatic.
- Lower risk, less human error, rapid value delivery.

CI/CD IN THE DEVOPS LIFECYCLE

WHY CI/CD MATTERS

- **Deliver software faster**
- **Improve quality & reliability**
- **Reduce risks & manual errors**
- **Increase collaboration**
- **Drive business value**

KEY TAKEAWAY CI/CD enables teams to build, test, and release better software continuously -- so users get value faster and with confidence.

What is CI/CD?

CI/CD (Continuous Integration / Continuous Delivery or Continuous Deployment) is an automated approach to build, test, and deliver software changes quickly and reliably.

CI – CONTINUOUS INTEGRATION

Automate build and test of code changes

1. CODE

Developers write and commit code to version control

2. INTEGRATE

Code is automatically integrated into the shared repository

3. BUILD

The code is compiled and dependencies are installed

4. TEST

Automated tests run to validate code quality and safety

CD – CONTINUOUS DELIVERY / DEPLOYMENT

Automate release and deployment of application

5. PACKAGE

Build artifacts are packaged and versioned

6. RELEASE

Artifacts are stored in a repository and released

7. DEPLOY

Automated deployment to test/staging/ production

8. MONITOR

Application is monitored for performance, errors, and feedback

KEY BENEFITS OF CI/CD

FASTER DELIVERY

Deliver features and fixes to users quickly and frequently.

HIGHER QUALITY

Automated testing catches issues early and improves reliability.

IMPROVED PRODUCTIVITY

Automation reduces manual work and lets teams focus on value.

BETTER COLLABORATION

Shared pipelines and visibility improve communication across teams.

REDUCED RISK

Consistent, repeatable processes reduce failures and support rollbacks.

COST EFFICIENCY

Fewer errors in production and faster delivery lower operational costs.

CI ensures the code is always ready to be released. | **CD** ensures the code is always ready to be delivered or deployed.

CONTINUOUS INTEGRATION OVERVIEW

CI is a DevOps practice where developers frequently integrate code changes into a shared repository, verified automatically by building and testing every change.

THE CONTINUOUS INTEGRATION PIPELINE

BEST PRACTICES FOR CI

- Commit small and atomic changes; push to a shared repo.
- Automate builds and tests for every commit.
- Keep the build fast (minutes, not hours).
- Maintain a reliable, isolated build environment.
- Make feedback visible and actionable; keep main always releasable.

WHY CI MATTERS

- **Detect Issues Early --** find bugs as soon as they're introduced.
- **Improve Quality --** automated tests run on every change.
- **Increase Collaboration --** frequent integration keeps teams aligned.
- **Accelerate Delivery --** a stable codebase enables faster releases.

KEY TAKEAWAY Continuous Integration builds trust in the codebase by ensuring every change is integrated, built, and tested automatically -- the foundation for Delivery and Deployment.

CONTINUOUS INTEGRATION OVERVIEW

Continuous Integration (CI) is a DevOps practice where developers frequently integrate code changes into a shared repository. Each integration is automatically built, tested, and validated early to detect issues quickly and improve software quality.

CI PIPELINE WORKFLOW

1 CODE COMMIT

Developers write code and commit changes to the version control system.

2 SOURCE CONTROL

Code is pushed to a shared repository (e.g., Git) which triggers the CI pipeline.

3 BUILD

The CI server checks out the code and compiles it. Dependencies are resolved and artifacts are created.

4 AUTOMATED TESTS

Automated tests (unit, integration, etc.) are executed to validate the changes.

5 QUALITY CHECKS

Static code analysis, code style, security scans, and other quality checks are performed.

6 RESULT

The build result is reported. If successful, the code is ready for the next stage (CD pipeline).

GOALS

- Detect integration issues early
- Improve code quality
- Increase team collaboration
- Reduce manual effort
- Deliver better software, faster

KEY BENEFITS

Early Bug Detection
Find and fix issues early in the development cycle.

Higher Code Quality
Consistent testing and checks ensure better code quality.

Faster Feedback
Developers get immediate feedback on their changes.

Improved Collaboration
Encourages frequent integration and shared ownership.

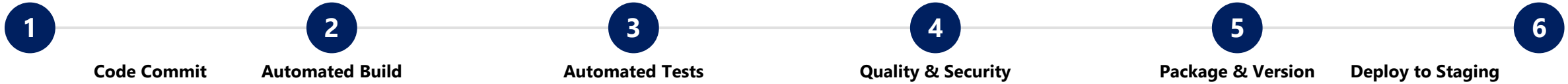
Reduced Risk
Small changes lower the risk of integration failures.

Increased Productivity
Automated processes save time and reduce manual work.

CONTINUOUS DELIVERY OVERVIEW

CD ensures code changes are always in a releasable state and can be deployed to production at any time -- extending CI with automated delivery to staging.

THE CONTINUOUS DELIVERY PIPELINE



KEY CHARACTERISTICS

- Automated pipeline from code commit to staging deployment.
- Every change is built, tested, and validated automatically.
- Always in a releasable state, ready for production.
- Manual approval before production release, when required.

WHY CONTINUOUS DELIVERY?

- Always ready to release.
- Reduced risk of changes.
- Faster time to market.
- Higher quality and reliability.

DELIVERY vs DEPLOYMENT

Practice	To Production
Continuous Delivery	Manual or gated
Continuous Deployment	Automatic

KEY TAKEAWAY Continuous Delivery ensures your software is always in a releasable state -- automated delivery to staging and rigorous validation let you release at any time with confidence.

CONTINUOUS DELIVERY OVERVIEW

Continuous Delivery (CD) is a software engineering practice where code changes are automatically built, tested, and prepared for release to production at any time.
Every change that passes all stages of the pipeline can be released with confidence.

THE CONTINUOUS DELIVERY PIPELINE

1. CODE COMMIT

Developers commit code changes to version control.

2. BUILD

The code is compiled and dependencies are resolved.

3. AUTOMATED TESTS

Automated tests (unit, integration, UI, etc.) are executed.

4. QUALITY GATES

Code quality, security scans, and other checks must pass.

5. STAGING DEPLOYMENT

The application is deployed to a staging environment.

6. PRODUCTION RELEASE

The application is ready to be released to production anytime.

KEY PRINCIPLE: Always keep the software in a deployable state. Any change that passes the pipeline can be released to production instantly.

WHAT CONTINUOUS DELIVERY ENABLES

FASTER DELIVERY
Deliver new features and fixes to users quickly and reliably.

HIGHER QUALITY
Automated testing and quality gates ensure better code quality.

REDUCED RISK
Small, frequent changes reduce risk and make releases more stable.

INCREASED AUTOMATION
Automated pipeline reduces manual work and human errors.

IMPROVED VISIBILITY
Full visibility into the delivery process and release readiness.

GREATER COLLABORATION
Encourages collaboration between development, QA, and operations.

CD PIPELINE FLOW

Code Commit → Build → Automated Tests → Quality Gates → Staging Deployment → Manual Approval (Optional) → Production Release → Monitoring & Feedback

CONTINUOUS DELIVERY = AUTOMATE EVERYTHING + DEPLOY ANYTIME

CONTINUOUS DEPLOYMENT OVERVIEW

Every change that passes all automated tests and quality gates is automatically deployed to production -- without manual intervention.

THE CONTINUOUS DEPLOYMENT PIPELINE

1. **Commit**
2. **Build**
3. **Tests**
4. **Quality Gates**
5. **Package**
6. **Auto Deploy**
7. **Verify & Monitor**

KEY CHARACTERISTICS

- **Fully Automated**
- **No Manual Gates**
- **Small, Safe Changes**
- **Monitoring & Feedback**
- **Culture of Automation**

CI vs CD vs CONTINUOUS DEPLOYMENT

Practice	What It Means	Deploy to Production
Continuous Integration (CI)	Integrate code frequently and run automated tests.	No (artifacts only)
Continuous Delivery (CD)	Always keep code in a releasable state.	Manual or scheduled release
Continuous Deployment	Automatically deploy every change that passes.	Automatic, no manual step

KEY TAKEAWAY Continuous Deployment delivers value to users automatically and continuously by combining automation, quality, and monitoring to the highest level.

CONTINUOUS DEPLOYMENT OVERVIEW

Continuous Deployment (CD) is an advanced DevOps practice where every code change that passes all automated tests and quality gates is automatically deployed to production.
No manual approvals. No waiting.
The result is fast, reliable, and consistent delivery of value to users.

THE CONTINUOUS DEPLOYMENT PIPELINE

1 CODE COMMIT
Developers commit code changes to version control.

2 BUILD
The code is compiled and dependencies are resolved.

3 AUTOMATED TESTS
Automated tests (unit, integration, UI, security, etc.) are executed.

4 QUALITY GATES
Code quality, security scans, and other checks must pass.

5 DEPLOY TO PRODUCTION
The application is automatically deployed to production.

6 VERIFY & MONITOR
The deployment is verified and monitored in real time.

KEY BENEFITS

FASTER TIME TO MARKET
Deliver features and fixes to users immediately.

HIGHER QUALITY
Automation and testing reduce defects and improve stability.

REDUCED RISK
Small, frequent changes minimize risk and simplify rollback.

INCREASED EFFICIENCY
End-to-end automation eliminates manual intervention.

BETTER VISIBILITY
Real-time monitoring ensures system health and performance.

GREATER COLLABORATION
Development, QA, and Operations work together seamlessly.

HOW IT WORKS

Code Commit → Integration → Build → Automated Tests → Quality Gates → Auto Deploy to Production → Monitor & Feedback

Continuous Feedback Loop

KEY PRINCIPLE

**AUTOMATE EVERYTHING.
DEPLOY EVERY CHANGE.**
If it's not automated, it's not continuous.
If it's not in production, it's not delivering value.

GOAL

Deliver value continuously by automatically deploying every change that meets quality and safety standards.

- Always releasable
- Always deployed
- Always monitored
- Always improving

GITHUB AND BITBUCKET OVERVIEW

GitHub and Bitbucket are leading source code management (SCM) platforms that help teams collaborate, manage code, and automate software delivery at scale.

GITHUB

- Cloud-based platform with the largest developer community.
- Vast open-source ecosystem and a rich integrations marketplace.
- Advanced security and code scanning; GitHub Actions for CI/CD.

BITBUCKET

- Designed for professional teams; strong security and access controls.
- Deep integration with Jira and other Atlassian tools.
- Scalable for enterprise teams with built-in Bitbucket Pipelines.

GITHUB vs BITBUCKET

Platform	Best For	CI/CD	Community	Hosting
GitHub	Open source, startups, developers	GitHub Actions	Very large	GitHub Cloud (SaaS)
Bitbucket	Enterprises, Atlassian-tool teams	Bitbucket Pipelines	Smaller, enterprise	Cloud / Data Center

COMMON USE CASES

Store & manage code | **Pull requests & reviews** | **Issue tracking** | **Automate & deploy**

KEY TAKEAWAY GitHub and Bitbucket both provide version control, collaboration, and automation -- this course uses GitHub and GitHub Actions throughout.

GitHub and Bitbucket Overview

GitHub

The world's leading AI-powered developer platform.

GitHub is a cloud-based platform built around Git. It enables developers to host code, collaborate, automate workflows, and build software together.

KEY FEATURES

Code Hosting
Unlimited public repositories and private repos

Collaboration
Pull requests, code reviews, mentions, and discussions

Actions / CI/CD
Automate build, test, and deploy pipelines with GitHub Actions

Integrations
Large marketplace with 3rd party tools and services

Security
Code scanning, dependency security, secret scanning

GITHUB WORKFLOW

1. Code
Developer writes and commits code

2. Push
Push code to GitHub repository

3. Pull Request
Create a pull request for code review

4. Review & Merge
Review, discuss and merge changes

5. Actions / CI/CD
Automated build, test and deployment

Bitbucket

The Git solution for professional teams.

Bitbucket is a Git-based platform designed for teams. It helps developers collaborate, manage code, and automate CI/CD in a secure environment.

KEY FEATURES

Code Hosting
Private and public repositories with Git

Collaboration
Pull requests, code reviews, comments, and approvals

Pipelines / CI/CD
Built-in CI/CD with Bitbucket Pipelines

Integrations
Integrates with Atlassian tools and other services

Security
IP allowlisting, 2FA, audit logs, and branch permissions

BITBUCKET WORKFLOW

1. Code
Developer writes and commits code

2. Push
Push code to Bitbucket repository

3. Pull Request
Create a pull request for code review

4. Review & Merge
Review, discuss and merge changes

5. Pipelines / CI/CD
Automated build, test and deployment

AT A GLANCE COMPARISON

	GitHub	Bitbucket
OWNER	Microsoft (since 2018)	Atlassian
BEST FOR	Open source, public projects, and large developer community	Teams, enterprises, and private projects within organizations
PUBLIC REPOS	Unlimited (Free plan)	
PRIVATE REPOS	Limited on Free plan (Unlimited on paid)	
CI/CD	GitHub Actions (Powerful & flexible)	Bitbucket Pipelines (Built-in & easy)
INTEGRATION	Large 3rd party ecosystem	Large 3rd party aclosystem
INTEGRATION		Seamless with Atlassian tools (Jira, Confluence)

SOURCE CODE MANAGEMENT WORKFLOW

SCM is the foundation of modern DevOps -- it provides version control, traceability, collaboration, and governance for software development teams.

THE SCM WORKFLOW (END-TO-END)

KEY BENEFITS

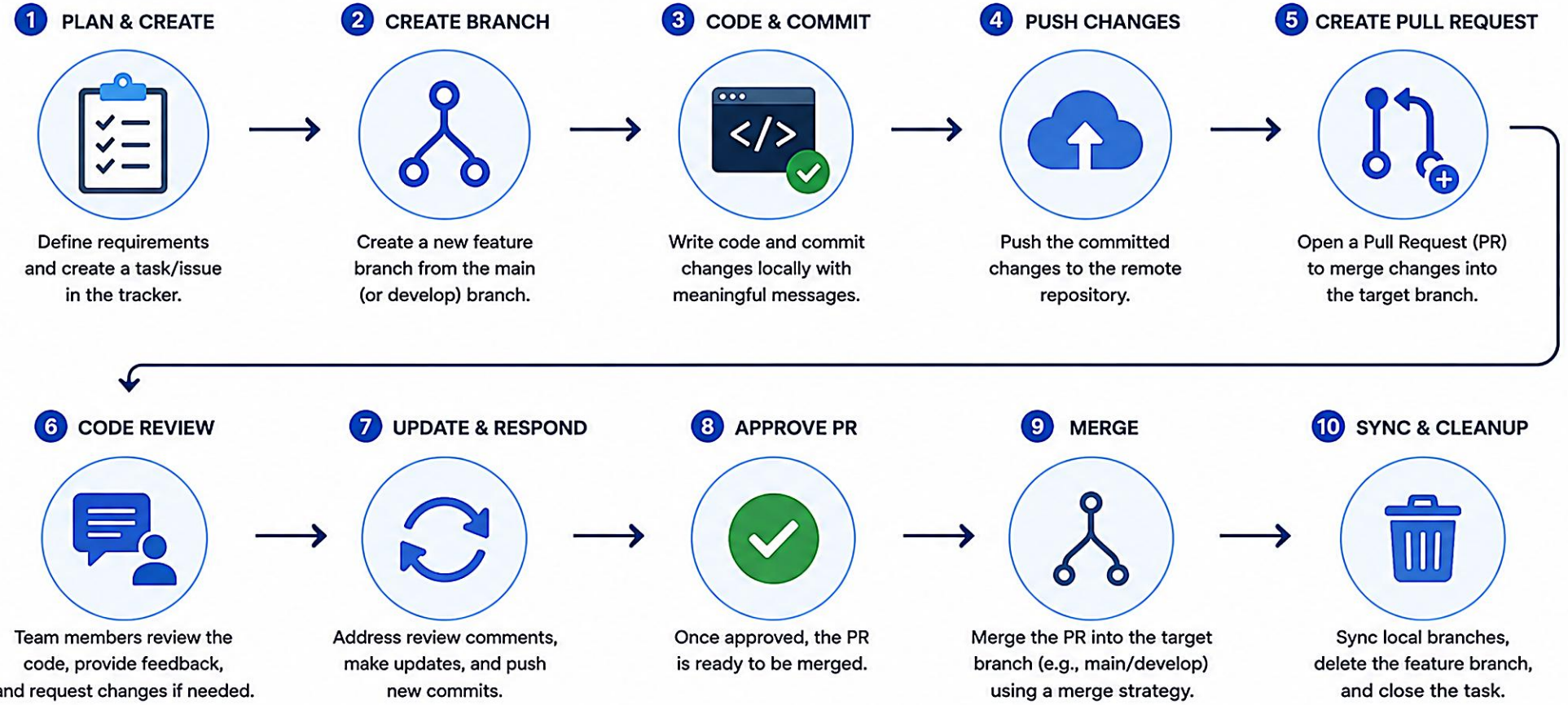
- Track every change with full history.
- Enable collaboration and clear code ownership.
- Improve code quality with reviews and automated tests.
- Faster, more reliable delivery with automated pipelines.

BEST PRACTICES

- Commit small, atomic changes with clear messages.
- Keep branches short-lived; review before you merge.
- Automate builds and tests; protect the main branch.
- Monitor and act on feedback continuously.

KEY TAKEAWAY A well-defined SCM workflow ensures traceability, quality, and automation -- forming the backbone of a successful CI/CD practice.

Source Code Management Workflow



Remote Repository
Central location for all code, branches, history, and collaboration.

Branching Strategy
Use branches to work in isolation and keep the main branch stable.

Collaboration
Reviews and discussions ensure code quality and team alignment.

Automation
CI/CD pipelines automatically build, test, and validate changes.

Traceability
Full history of changes, who made them, and why they were made.

BRANCHING STRATEGIES

Branching strategies help teams manage code changes, release cycles, and collaboration effectively -- choose the strategy that fits your team size and release frequency.

1. Git Flow

Defined branches for features, releases, and hotfixes. Well-structured and predictable; best for large teams and complex, scheduled releases.

2. Trunk-Based Development

Develop directly on main with short-lived feature branches. Simple, fast feedback; requires strong CI/CD discipline.

3. GitHub Flow

Branch, commit, open PR, review, merge. Very simple; ideal for continuous deployment, web apps, and small agile teams.

WHEN TO USE WHICH STRATEGY?

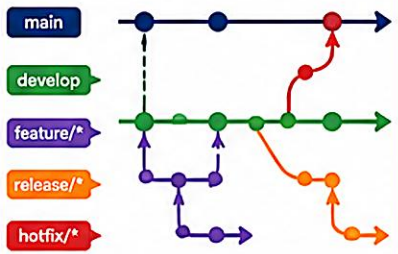
Strategy	Team Size	Release Frequency	Complexity	Best Suited For
Git Flow	Large	Scheduled (weeks/months)	High	Enterprise, regulated products
Trunk-Based Development	Small to Large	Very frequent (daily)	Medium to High	Teams with strong CI/CD
GitHub Flow	Small to Medium	Continuous	Low to Medium	Web apps, startups, SaaS

KEY TAKEAWAY Choose a branching strategy that matches your team's workflow and goals -- start simple, automate everything, and evolve as your product and team grow.

Branching Strategies

Different branching strategies help teams manage code changes, releases, and collaboration effectively.

1. Git Flow



How it Works

Features branch off **develop**.
Release branches prepare for production.
Hotfixes branch off **main**.

Pros

- Well-structured for releases
- Supports parallel development
- Good for large projects

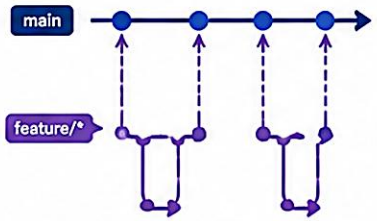
Cons

- Complex
- Overhead for small teams

Best For

Large teams, enterprise projects with scheduled releases.

2. GitHub Flow



How it Works

Create a feature branch from **main**.
Make changes and open a Pull Request.
After review, merge to **main** and deploy.

Pros

- Simple and lightweight
- Encourages continuous deployment
- Great for smaller teams

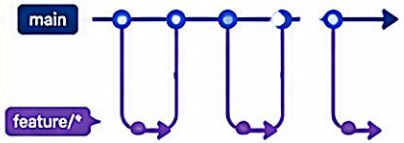
Cons

- Less suitable for complex releases
- No built-in release management

Best For

Small to medium teams, continuous delivery environments.

3. Trunk-Based Development



How it Works

Develop directly on **main** using very short-lived feature branches.
Integrate changes frequently.

Pros

- Fast integration
- Reduces merge conflicts
- High visibility

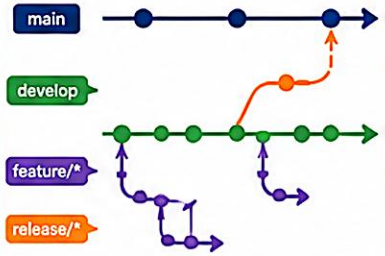
Cons

- Requires strong automation
- Not ideal for very large changes

Best For

Teams with strong CI/CD practices and automated testing.

4. Release Branch



How it Works

Features branch off **develop**.
A release branch is created for final testing and fixes, then merged into **main**.

Pros

- Stable release preparation
- Supports parallel feature work
- Clear separation of concerns

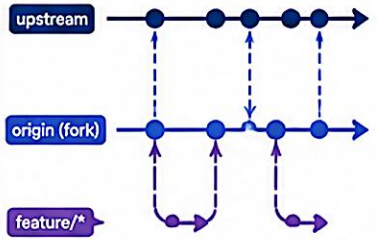
Cons

- More branches to manage
- Can become complex

Best For

Projects with regular releases and dedicated stabilization phase.

5. Forking Workflow



How it Works

Fork the **upstream** repository.
Create feature branches in your fork.
Submit Pull Requests to upstream.

Pros

- Safe contribution model
- No direct write access needed
- Ideal for open source

Cons

- Keeping fork up to date
- PRs can take time to merge

Best For

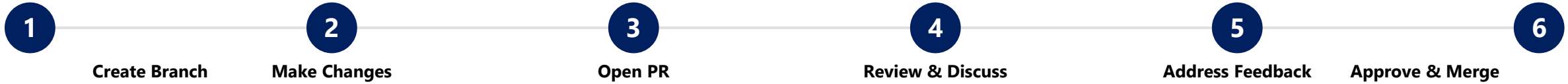
Open source projects, external contributors.

- **main / master** — Production-ready code
- **develop** — Integration branch
- **feature/*** — New feature development
- **release/*** — Release preparation
- **hotfix/*** — Urgent fixes for production
- → Merge
- --→ Pull / Sync

PULL REQUESTS AND REVIEWS

Pull Requests (PRs) are the heart of collaborative development -- a structured way to propose changes, discuss improvements, and ensure code quality before merging.

THE PULL REQUEST WORKFLOW



BEST PRACTICES FOR EFFECTIVE PRs

- Keep PRs small and focused; write clear titles and descriptions.
- Link related issues; follow coding standards.
- Add tests for new functionality; ensure all checks pass.
- Respond to comments promptly; rebase or update regularly.

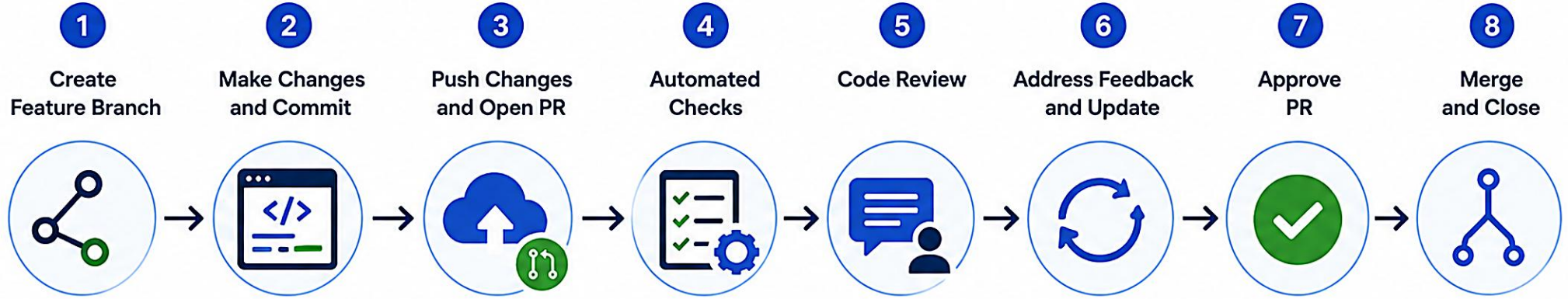
CODE REVIEW CHECKLIST

- Does the code solve the problem? Any bugs or edge cases?
- Is it easy to read, with good naming and no duplication?
- Are tests adequate and passing? Is error handling sufficient?
- Any performance or security concerns?

KEY TAKEAWAY Pull Requests and Reviews ensure high-quality code through collaboration, feedback, and automation -- reducing risk and improving team productivity.

Pull Requests and Reviews

A collaborative process to propose changes, review code, and merge high-quality code.



1. **Create Feature Branch** — Developer creates a feature branch from the target branch (e.g., main).
2. **Make Changes and Commit** — Developer implements changes and commits them to the feature branch.
3. **Push Changes and Open PR** — Developer pushes the branch to the remote and opens a Pull Request (PR).
4. **Automated Checks** — CI/CD runs automated checks, builds, and tests. Results are reported on the PR.
5. **Code Review** — Reviewers examine the code, provide feedback, and suggest improvements.
6. **Address Feedback and Update** — Developer addresses feedback, makes updates, and pushes new commits.
7. **Approve PR** — Once all feedback is addressed, reviewers approve the Pull Request.
8. **Merge and Close** — PR is merged into the target branch. The feature branch is deleted, and the PR is closed.

Benefits

Code Quality
Peer reviews catch bugs, reduce defects, and improve code quality.

Knowledge Sharing
Team members share knowledge and follow best practices.

Collaboration
Encourages discussion and collaboration across the team.

Early Detection
Automated checks detect issues early in the development cycle.

Traceability
Provides a history of changes, discussions, and decisions.

Continuous Improvement
Regular reviews lead to continuous improvement in code and processes.

INTRODUCTION TO GITHUB ACTIONS

GitHub Actions is a powerful CI/CD and automation platform built into GitHub -- build, test, and deploy your code using workflows triggered by events.

KEY BENEFITS

- **Automate Anything --** build, test, deploy, release, and more.
- **Integrated & Secure --** built into GitHub with fine-grained permissions and secrets.
- **Cost-Effective --** generous free tier for public and private repos.
- **Flexible & Extensible --** GitHub-hosted or self-hosted runners; thousands of reusable actions.

HOW GITHUB ACTIONS WORKS

EXAMPLE WORKFLOW -- ci.yml

```yaml
name: CRM CI
on:
  pull_request:
  push:
    branches: [main]
jobs:
  verify:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: '21'
          cache: maven
      - run: ./mvnw -B clean verify
```

COMMON TRIGGER EVENTS

push | pull_request | workflow_dispatch | schedule | release

KEY TAKEAWAY GitHub Actions brings automation to your repository -- enabling CI/CD, improving quality, and accelerating delivery for the CRM pipeline.

Introduction to GitHub Actions

GitHub Actions is a CI/CD platform that lets you automate your build, test, and deployment workflows right in your GitHub repository.

What is GitHub Actions?

A CI/CD service integrated with GitHub to automate your software workflows.

Workflow as Code

Define your workflows in YAML files and store them in your repository.

Built-in & Marketplace Actions

Use pre-built actions or create custom ones to speed up automation.

Secure & Reliable

Runs in isolated environments with secrets management and access control.

Scalable & Flexible

From small projects to enterprise applications, automate any workflow.

How It Works

1. Event Trigger

A workflow is triggered by an event (push, pull request, schedule, etc.).

2. GitHub Receives Event

GitHub detects the event and starts the workflow.

3. Run Workflow

Jobs defined in the workflow are queued and executed in GitHub-hosted runners.

4. Execute Jobs

Steps run in order to build, test, lint, or deploy your application.

5. Workflow Complete

Results are reported. You can see logs, artifacts, and status in the Actions tab.

Key Features

Automate CI/CD pipelines	Build, test, and deploy automatically
Event-driven	Respond to pushes, PRs, issues, schedules, and more
Reusable	Use and share workflows and actions
Secure	Secrets, permissions, and environment protection
Multi-platform	Run on Ubuntu, Windows, macOS, and containers

Example Workflow

ci.yml

```yaml
name: CI Pipeline

on:
  push:
    branches: [ main ]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm install
      - run: npm test
```

TRY IT YOURSELF -- EX 1: PIPELINE POLICY

Reinforces: which GitHub Actions events should run which jobs -- decide the policy before writing any YAML.

REFERENCE MATRIX (notes/lab43-pipeline-policy.md)

Event	Verify	Package JAR + SHA-256
pull_request	Yes	No (typical)
push to main	Yes	Yes
tag v*	Yes	Yes

STEPS

- Step 1 -- Fill a matrix: event -> jobs (verify always; package on main/tags; deploy later, not yet).
- Step 2 -- Apply leadership's rule: PRs get fast feedback; main/tags get stronger gates; deploy credentials never live in Git.
- Step 3 -- Note that synthetic fixtures (CUS-1001, lab-request-001) may appear only in test evidence, never real data.

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 1: exercises/exercise-01-pipeline-policy.md (workspace: examples/module-43-exercises).

PIPELINE ARCHITECTURE

A well-architected pipeline automates the flow of code from commit to deployment through defined stages, environments, and quality gates.

HIGH-LEVEL PIPELINE FLOW

KEY PRINCIPLES

- **Automation First**
- **Consistency**
- **Security by Design**
- **Quality Gates**
- **Visibility & Feedback**

CROSS-CUTTING CONCERNS

- Version control, secrets management, artifact/container registry, CI/CD platform, monitoring, and access control (RBAC) apply across every stage.

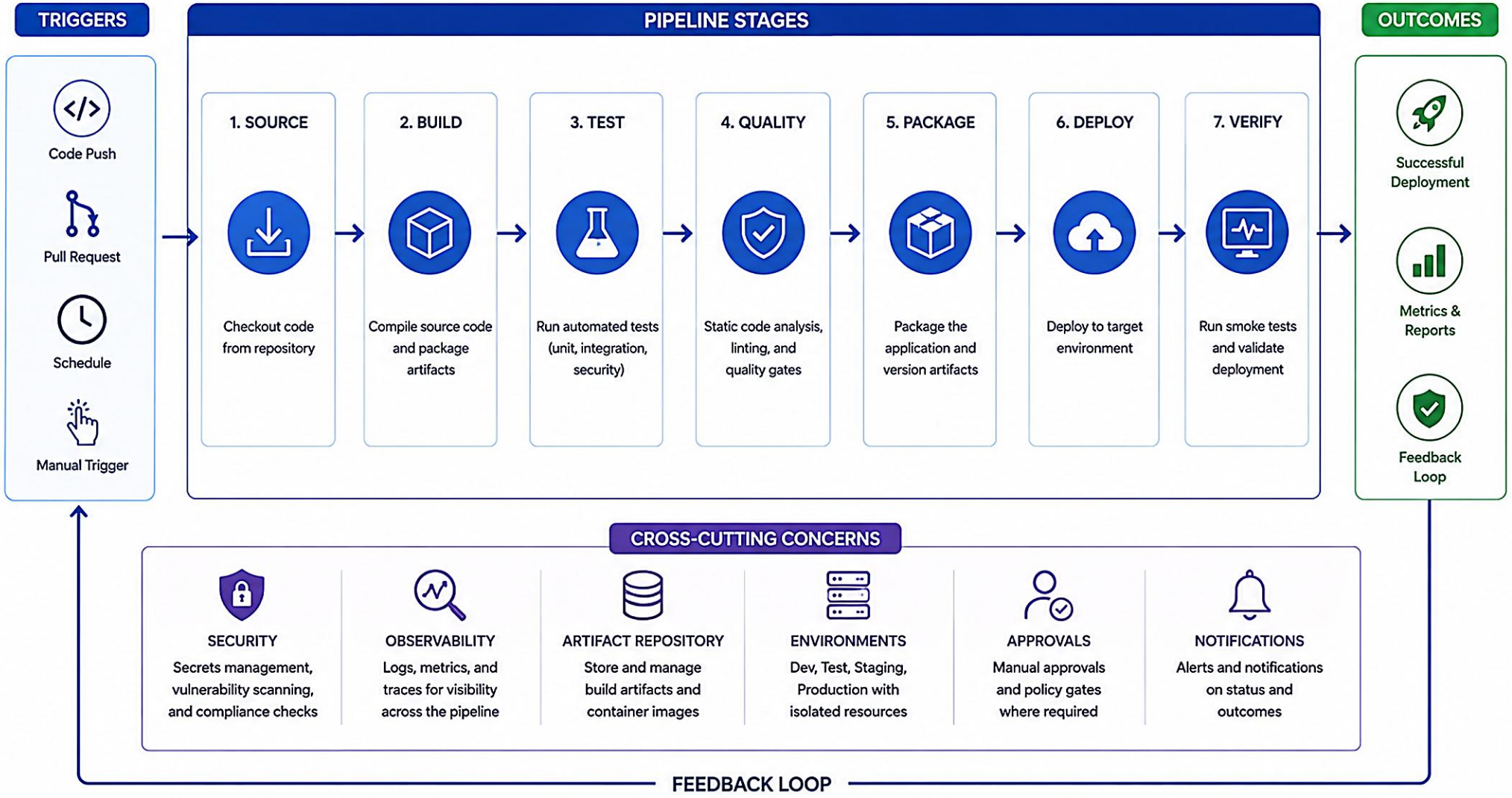
PIPELINE ARCHITECTURE COMPONENTS

Component	What It Is
Triggers	Events that start the pipeline (push, PR, schedule, release).
Stages / Jobs	Logical phases (build, test, scan, deploy), run in order or parallel.
Steps	Individual commands or actions executed in a job.
Artifacts	Build outputs, reports, and packages shared across stages.
Environments / Runners	Deploy targets (dev/test/staging/prod) and the machines that run jobs.

KEY TAKEAWAY A strong pipeline architecture ensures reliable, secure, and fast delivery of high-quality software with full visibility and control.

Pipeline Architecture

A CI/CD pipeline is a series of automated stages that build, test, and deliver software reliably.



PIPELINE CONFIGURATION FILE

GitHub Actions uses YAML configuration files to define workflows. These files live in the .github/workflows/ directory of your repository.

EXAMPLE: .github/workflows/ci.yml

```yaml
name: CRM CI
on:
  pull_request:
  push:
    branches: [main]
    tags: ["v*"]
jobs:
  verify:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: "21"
          cache: maven
      - name: Verify
        run: ./mvnw -B clean verify
      - name: Upload reports
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: test-reports
          path: "**/target/surefire-reports/**"
```

YAML KEY ELEMENTS

Element	What It Defines
name	Name of the workflow.
on	Events that trigger the workflow.
jobs	Collection of jobs that run in the workflow.
steps	Sequence of tasks executed in a job.
uses / run	Reuses an action, or runs a shell command.
env / secrets	Environment variables and protected secrets.

BEST PRACTICES

- Commit workflow files to version control; keep them DRY.
- Pin action versions (e.g., @v4) for stability.
- Use descriptive job/step names; store secrets in GitHub Secrets.
- Use environments for deployment approvals.

KEY TAKEAWAY A well-structured pipeline configuration file is the foundation of reliable, repeatable, automated CI/CD delivery.

Pipeline Configuration File

Pipelines are defined as code in a YAML file (e.g., .github/workflows/ci.yml) in your repository.

ci.yml

```yaml
name: CI Pipeline
on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'

      - name: Install dependencies
        run: npm ci

      - name: Run tests
        run: npm test
```

1 Workflow Name
A friendly name for your pipeline.

2 Triggers (on)
Defines events that trigger the workflow (push or pull request on main branch).

3 Jobs
A workflow is made up of one or more jobs that run in parallel or sequentially.

4 Runner Environment
Specifies the environment to run the job.

5 Steps
A job is a series of steps executed in order.

6 Actions
Reuse actions from the GitHub Marketplace or your own custom actions.

7 Run Commands
Execute shell commands in the runner.

Key Elements

name
Name of the workflow.

on
Events that trigger the workflow.

jobs
Container for one or more jobs.

runs-on
Runner environment (OS or self-hosted).

steps
Ordered list of tasks in a job.

uses
Reference to an action to run a reusable task.

run
Run shell commands in the runner.

File Location

.github/workflows/ci.yml

Benefits

- Version controlled with your code
- Reviewable and collaborative
- Consistent and reproducible
- Auditable and traceable

Tips

- Use meaningful names for readability.
- Keep steps small and focused.
- Leverage reusable actions.
- Store secrets in GitHub Secrets.

TRY IT YOURSELF -- EX 2: PLAN THE VERIFY JOB

Reinforces: the YAML structure you just saw -- plan the JDK 21 verify job without skipping tests.

STEPS (notes/lab43-java21-verify.md)

Step	What To Do
1 -- Setup	List the Actions steps: checkout, setup-java Temurin 21 with Maven cache, ./mvnw -B clean verify.
2 -- Reports	Plan to upload Surefire/Failsafe reports even on failure, using if: always().
3 -- Failure Drill	Write how you will intentionally break one test, observe CI red, then restore (plan only).
4 -- Local Habit	Note the local preflight: java -version shows 21; ./mvnw -v before every push.

Preflight commands to record

```
java -version
./mvnw --version 2>/dev/null || mvn -version
./mvnw -B -ntp clean verify
```

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 2: exercises/exercise-02-java21-verify.md.

PIPELINE EXECUTION FLOW

The pipeline execution flow shows how a GitHub Actions workflow runs from trigger to completion, executing jobs and steps to deliver value.

END-TO-END PIPELINE EXECUTION FLOW

KEY POINTS

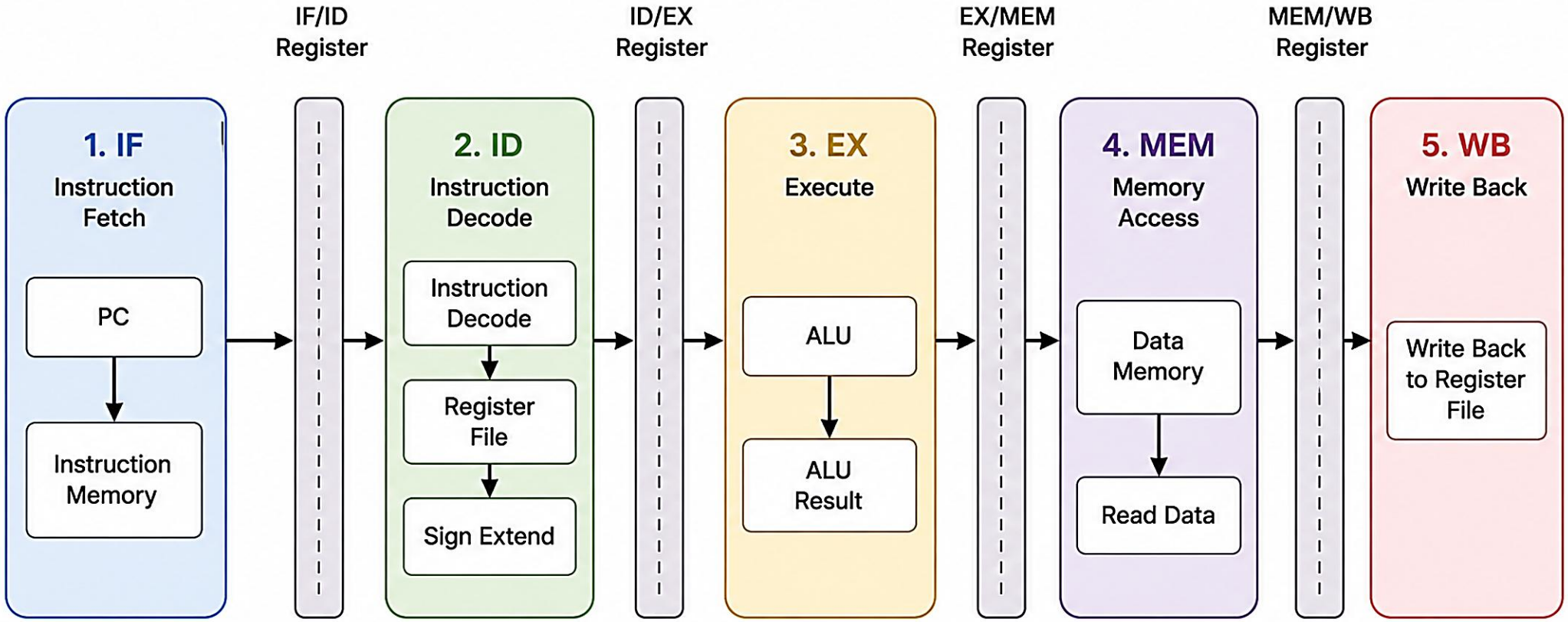
- A workflow run is created for every trigger.
- Jobs run on runners and can run in parallel; steps execute in order within each job.
- Results, artifacts, logs, and summaries are reported back in real time.
- The final status of every job determines overall success or failure.

COMMON STATUSES

- **Success --** completed successfully.
- **Failure --** one or more jobs failed.
- **Skipped --** condition not met.
- **In Progress / Waiting --** currently running, or waiting on a runner or dependency.

KEY TAKEAWAY The pipeline execution flow ensures that code moves through a predictable, automated path -- from event to high-quality delivery.

Pipeline Execution Flow



AUTOMATED BUILDS

Automated builds compile source code, resolve dependencies, run static checks, and produce build artifacts -- consistently and repeatably, every time code changes.

KEY BENEFITS

- **Consistency --** the same steps run every time, in the same environment.
- **Speed --** faster feedback to developers on every code change.
- **Quality --** early detection of errors and quality issues.
- **Traceability --** build history and artifacts are easy to track and audit.

BUILD OUTPUTS (ARTIFACTS)

Executable JAR | **Test Reports (JUnit)** | **Coverage & Logs**

BEST PRACTICES

- Keep builds fast; fail on any error. Cache dependencies. Use versioned actions (@v4) and reproducible build scripts.

EXAMPLE: MAVEN BUILD WORKFLOW

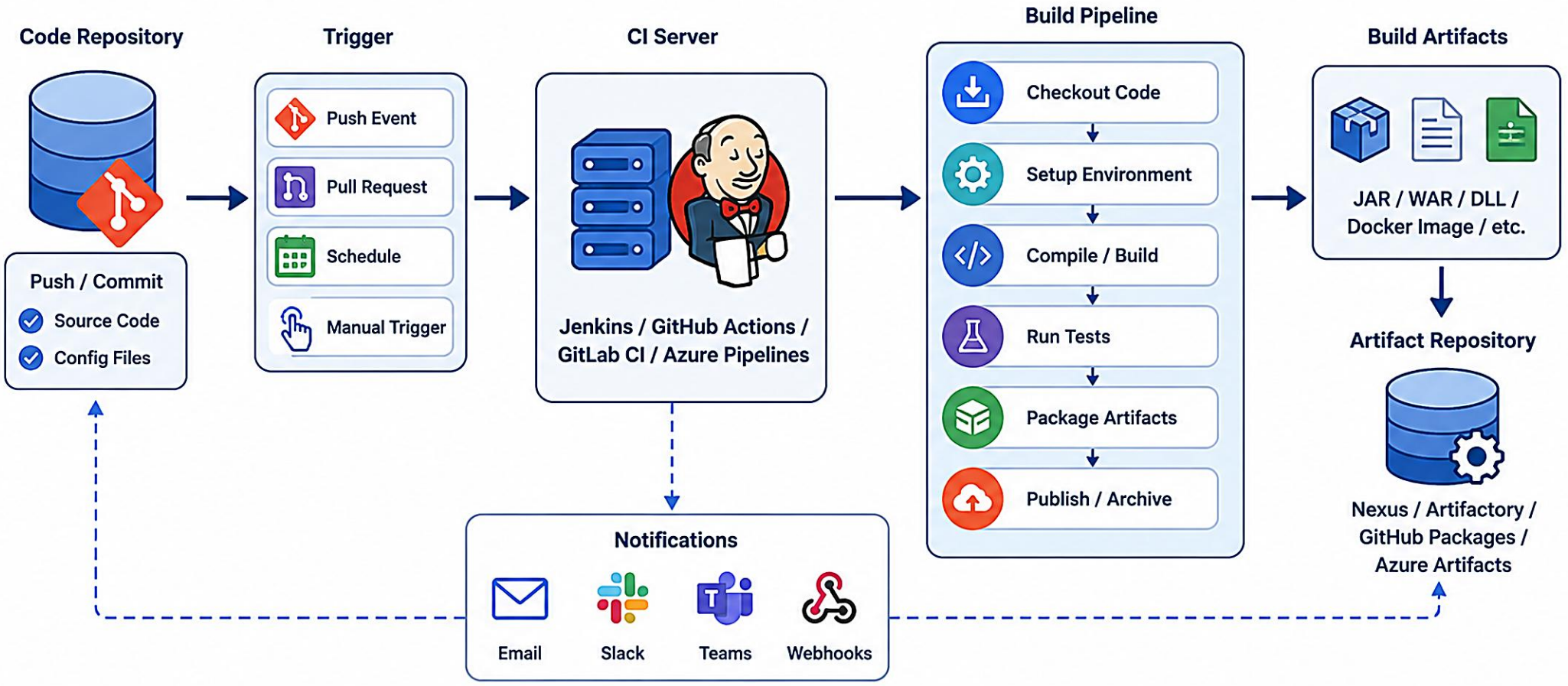
```yaml
name: Java Maven Build
on:
  push:
    branches: [main, develop]
  pull_request:
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: '21'
          cache: maven
      - run: ./mvnw -B clean verify
```

COMMON TRIGGERS

push | **pull_request** | **workflow_dispatch** | **schedule**

KEY TAKEAWAY Automated builds provide a reliable foundation for CI/CD by ensuring every change is compiled, tested, and validated consistently.

Automated Builds



Key Benefits

- Faster Feedback
- Consistent & Repeatable
- Early Issue Detection
- High Quality Artifacts
- Improved Productivity
- Reduced Costs

AUTOMATED UNIT TESTING

Automated unit testing runs tests as part of the CI pipeline to catch defects early, prevent regressions, and ensure code quality with every change.

KEY BENEFITS

- **Catch Defects Early --** find issues before code reaches production.
- **Prevent Regressions --** new changes don't break existing functionality.
- **Improve Confidence --** safer releases with reliable, repeatable tests.
- **Better Coverage --** track coverage and identify gaps.

EXAMPLE: MAVEN + JUNIT WORKFLOW

```
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { java-version: '21', cache: maven }
      - run: ./mvnw -B clean test
      - if: always()
        uses: actions/upload-artifact@v4
        with: { name: test-reports, path: '**/target/surefire-reports/**' }
```

EXAMPLE TEST REPORT SUMMARY

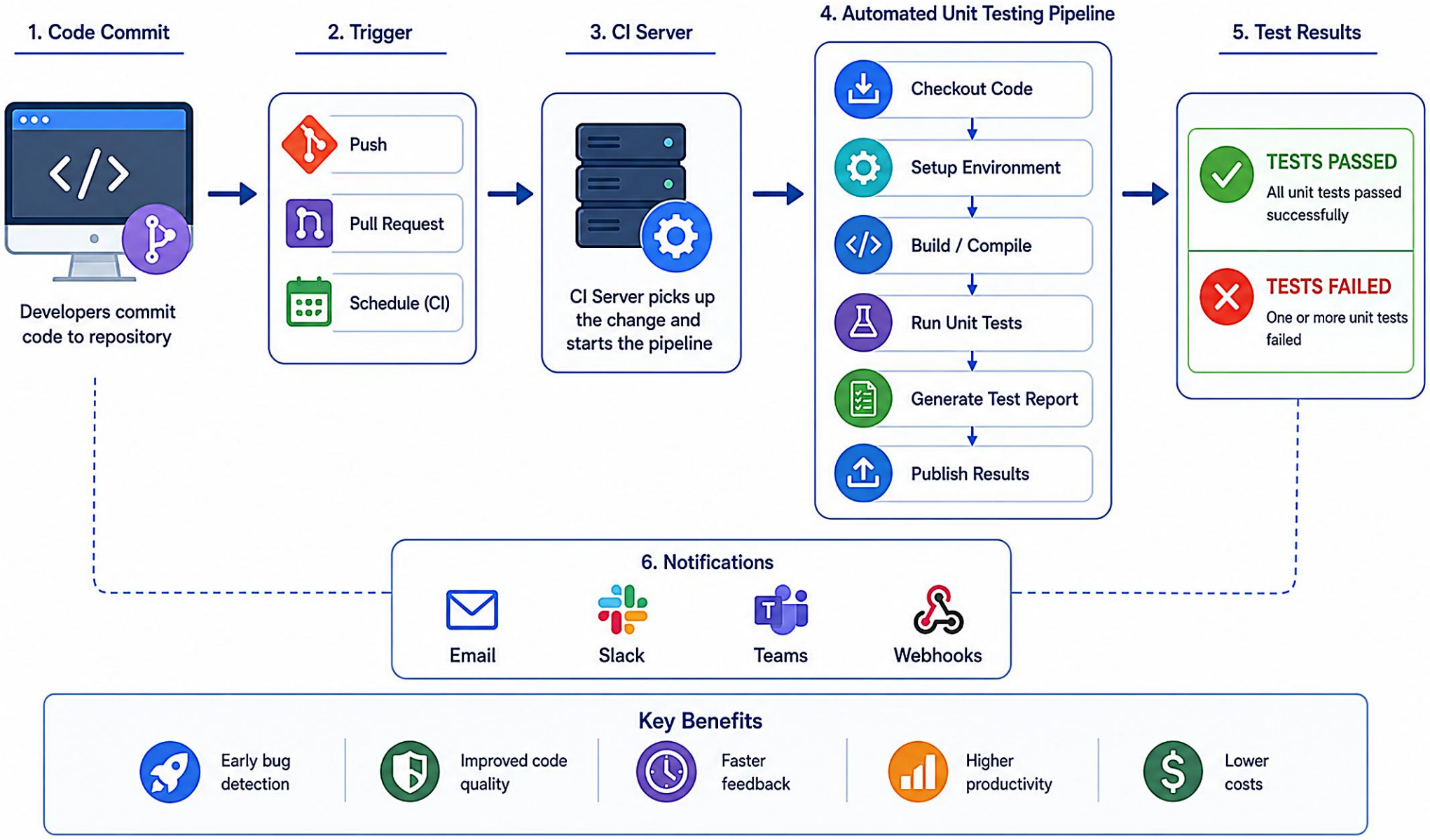
Total Tests	Passed	Failed	Skipped
128	118 (92.2%)	6 (4.7%)	4 (3.1%)

BEST PRACTICES

- Write small, independent tests; follow Arrange-Act-Assert.
- Maintain high coverage for critical logic; mock external dependencies.
- Keep tests fast and deterministic; fix failures immediately.

KEY TAKEAWAY Automated unit testing in the pipeline ensures high code quality, reduces defects, and builds confidence in every release.

Automated Unit Testing



AUTOMATED SECURITY SCANNING

Automated security scanning runs in the CI pipeline to identify vulnerabilities, misconfigurations, and secrets before code is merged or deployed -- shifting security left.

KEY BENEFITS

- **Find Issues Early --** catch vulnerabilities and secrets before production.
- **Reduce Risk --** lower the cost and impact of security issues.
- **Enforce Standards --** apply security policies and gates automatically.
- **Build Trust --** deliver secure software faster and with confidence.

EXAMPLE: SECURITY SCAN JOB

```yaml
jobs:
  security-scan:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      security-events: write
    steps:
      - uses: actions/checkout@v4
      - uses: github/codeql-action/init@v3
        with: { languages: java }
      - uses: github/codeql-action/analyze@v3
```

COMMON SECURITY TOOLS

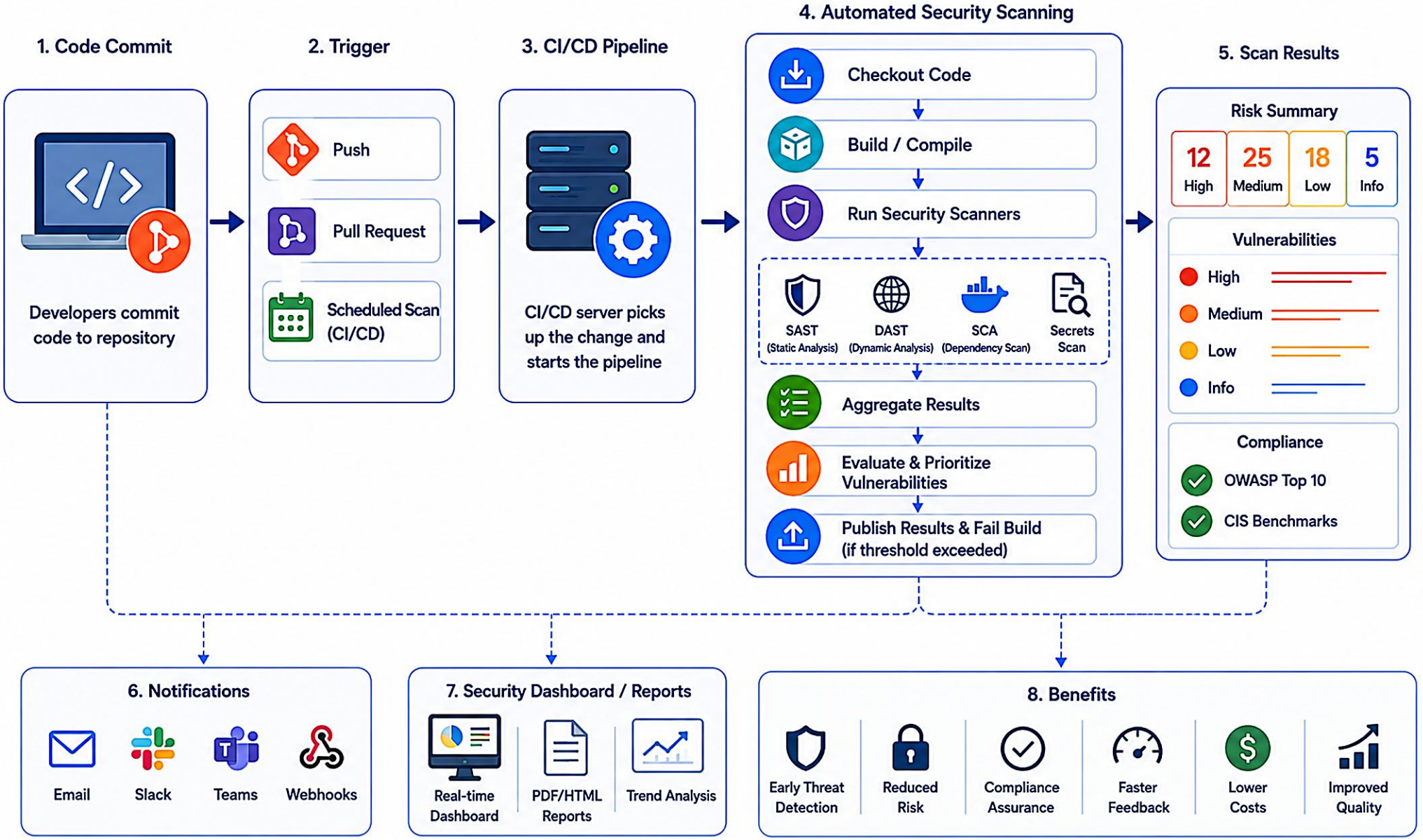
Tool	Purpose
CodeQL	Static application security testing (SAST) by GitHub.
OWASP Dependency-Check	Open-source dependency vulnerability scanning.
Gitleaks	Detects secrets and sensitive data in code.
Checkov	Infrastructure-as-Code security scanning.

BEST PRACTICES

- Scan on every push/PR; fail fast on critical/high severity; keep dependencies updated; review findings regularly.

KEY TAKEAWAY Automated security scanning in CI ensures vulnerabilities, secrets, and misconfigurations are detected early -- enforcing security by default.

Automated Security Scanning



AUTOMATED PACKAGING

Automated packaging converts the built application into a versioned artifact that can be deployed to any environment consistently and reliably.

KEY BENEFITS

- **Consistency --** the same steps every time produce the same artifact.
- **Reproducibility --** artifacts can be rebuilt from the same source.
- **Traceability --** versioned artifacts are easy to track and audit.
- **Reliability --** easier, safer rollback using previous artifacts.

BEST PRACTICES

- Package once; never rebuild in a later deploy step.
- Attach a SHA-256 checksum and the commit SHA for traceability.
- Keep artifacts immutable; store them in a secure repository.

PACKAGE ONCE + CHECKSUM (LAB 43 PATTERN)

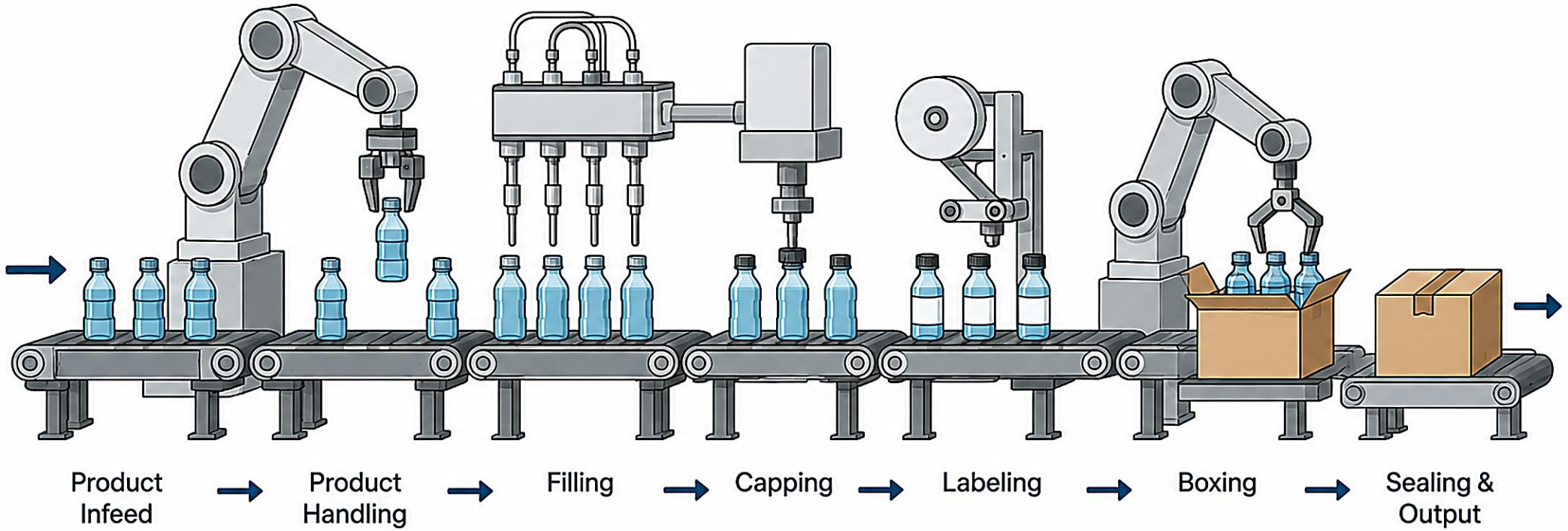
```
package:
  needs: verify
  if: github.ref == 'refs/heads/main' ||
      startsWith(github.ref, 'refs/tags/')
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-java@v4
      with: { java-version: '21', cache: maven }
    - run: |
        ./mvnw -B -DskipTests package
        sha256sum target/*.jar > target/SHA256SUMS
        echo "commit=${GITHUB_SHA}" >> target/SHA256SUMS
```

ARTIFACT METADATA

- version
- commit (GITHUB_SHA)
- branch
- build number
- timestamp

KEY TAKEAWAY Automated packaging ensures every build produces a reliable, versioned artifact tied to its commit -- never rebuilt, only promoted.

Automated Packaging



TRY IT YOURSELF -- EX 3: PACKAGE-ONCE IDENTITY

Reinforces: why the JAR verified in CI must be the exact JAR promoted later -- no silent rebuilds.

STEPS (notes/lab43-immutable-jar.md)

Step	What To Do
1 -- Steps	Outline: package once, write SHA256SUMS, record GITHUB_SHA, upload the artifact.
2 -- Reference	Note that Lab 44 promotes this identity -- rebuilding silently on the deploy agent breaks the chain.
3 -- Example Lines	Draft example checksum-file lines (fake hashes are fine) including the commit id.
4 -- Anti-Pattern	Name one anti-pattern: packaging differently in deploy than in CI.

Example SHA256SUMS lines to draft

```
a1b2c3d4e5f6...  crm-1.2.0.jar
commit=8f3a91c2b7de4f10aa55c6d9e2b1f0a3c4d5e6f7
run=147
```

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 3: exercises/exercise-03-immutable-jar.md.

TRY IT YOURSELF -- EX 4: FILL CI.YML TODOS

Reinforces: verify + package-once mechanics together -- fill in every blank before Lab 43 (starter, not a solution).

Fill in EVERY _____ (STARTER -- see lab-43-solution) **(notes/lab-43-workflow-todos.md)**

```
on:
  pull_request:
  push:
    branches: [main]
    tags: ["v*"]
jobs:
  verify:
    steps:
      - uses: actions/setup-java@v4
        with: { java-version: "_____" }
      - run: _____ -B clean verify
  package:
    needs: verify
    if: _____
    steps:
      - run: sha256sum target/*.jar > _____
```

#	What To Do
1	Fill java-version with "21" and the runner command with ./mvnw.
2	Fill the package if: with the main/tags condition from Exercise 1.
3	Fill the checksum output path with target/SHA256SUMS.
4	Add a YAML comment: # secrets via GitHub Actions secrets -- never hardcode.

KEY TAKEAWAY TRY IT NOW -- This is a STARTER, not a solution: fill every blank in exercises/exercise-04-workflow-todos.md before Lab 43.

ENVIRONMENT DEPLOYMENT WORKFLOW

The environment deployment workflow promotes safe, repeatable, reliable releases by deploying artifacts through multiple environments using approvals and quality gates.

DEPLOYMENT FLOW OVERVIEW

1. **Merge & Package**
2. **Deploy to Dev**
3. **QA Validation**
4. **Approval**
5. **Deploy to Staging**
6. **Deploy to Prod**
7. **Post-Deploy Checks**

ENVIRONMENTS

- **Development --** for developers to integrate and test changes.
- **QA --** automated and manual testing by the QA team.
- **Staging / UAT --** production-like environment for final validation.
- **Production --** the live environment serving end users.

APPROVAL GATES

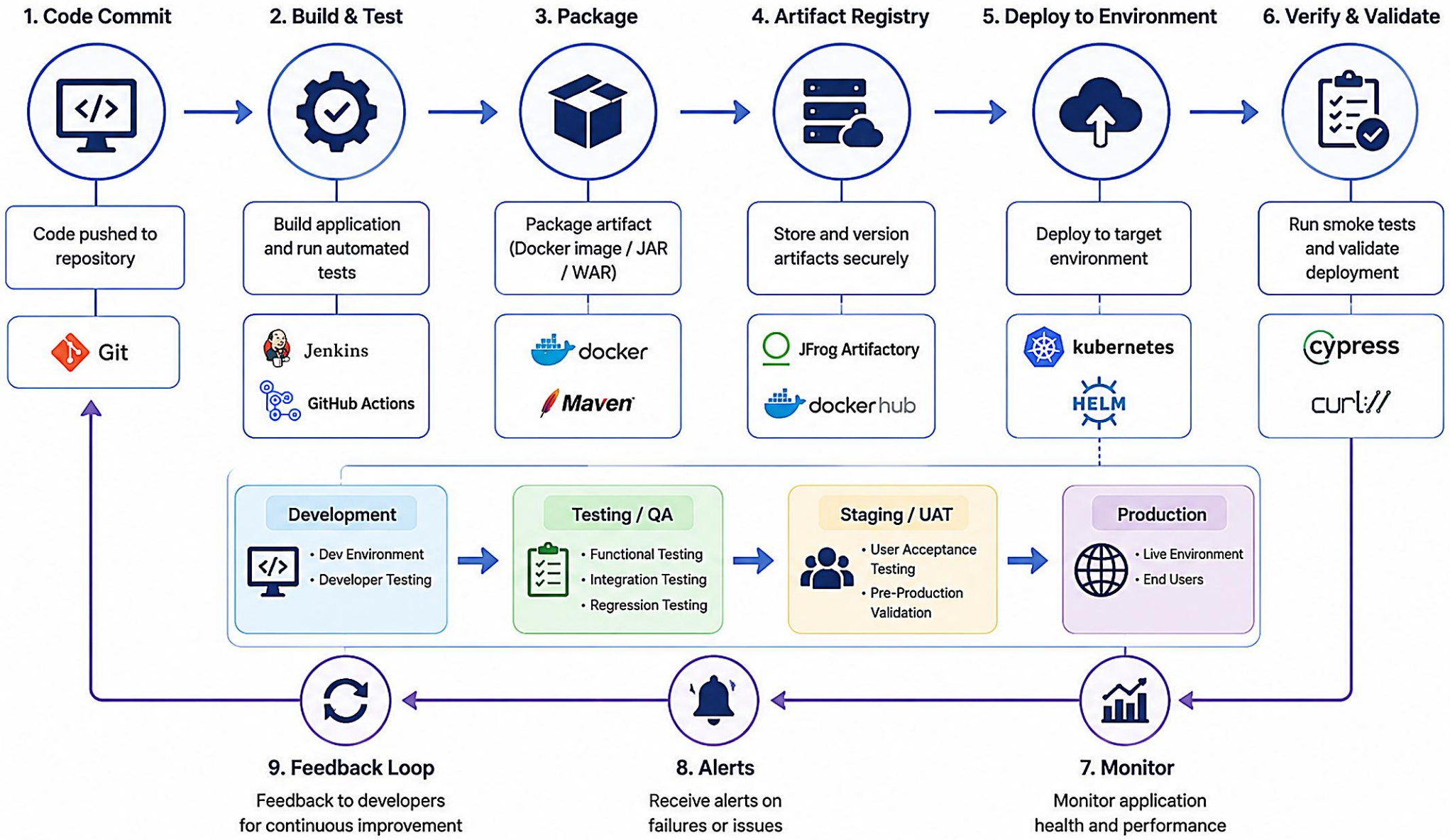
Stage	Approval By
QA -> Staging	QA Lead / Product Owner
Staging -> Production	Release Manager

DEPLOYMENT STRATEGIES

Rolling Update | **Blue/Green** | **Canary Release**

KEY TAKEAWAY A structured environment deployment workflow ensures every change is validated, approved, and released safely -- delivering value with confidence and minimal risk.

Environment Deployment Workflow



STAGING DEPLOYMENT

Staging is a production-like environment used to validate the release candidate before it goes to production -- catching integration and configuration issues early.

KEY BENEFITS

- **Risk Reduction --** detect issues before they impact real users.
- **Confidence --** ensure the build is stable and ready for production.
- **Realistic Validation --** an environment that mirrors production.

STAGING ENVIRONMENT CHARACTERISTICS

Mirrors Production | **Sanitized Data** | **Access & Monitoring**

BEST PRACTICES

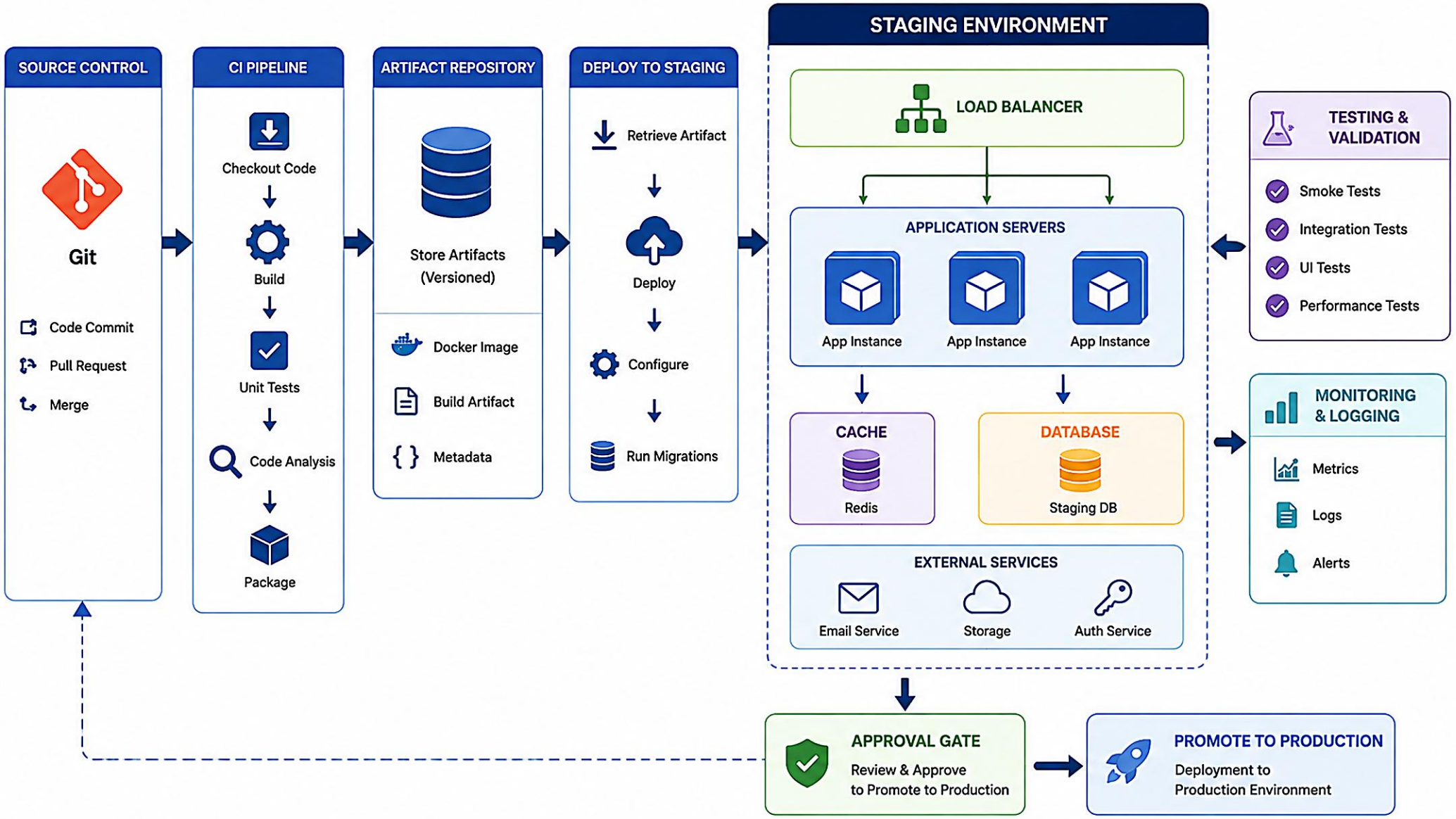
- Keep staging as close to production as possible; run smoke and health checks after every deployment; get explicit approval before promoting.

EXAMPLE: DEPLOY TO STAGING

```
deploy-staging:
  needs: package
  runs-on: ubuntu-latest
  environment:
    name: staging
    url: https://staging.crm.example.com
  steps:
    - uses: actions/checkout@v4
    - uses: actions/download-artifact@v4
      with: { name: crm-jar }
    - name: Deploy to Staging
      run: ./scripts/deploy.sh staging
    - name: Smoke test
      run: curl -f https://staging.crm.example.com/health
```

KEY TAKEAWAY Staging is your safety net -- it ensures what goes to production is tested, validated, and ready for your users.

Staging Deployment



PRODUCTION DEPLOYMENT

Production deployment releases the verified, approved software to end users -- following a controlled, automated, and observable process.

KEY BENEFITS

- **Reliability --** deliver stable, high-quality releases to users.
- **Security --** enforce approvals, access controls, and scans.
- **Visibility --** real-time monitoring, logs, and audit trails.
- **Rollback Ready --** quickly roll back to the last known-good version.

DEPLOYMENT STRATEGIES

Rolling Update **Blue/Green** **Canary Release**

DEPLOYMENT SAFEGUARDS

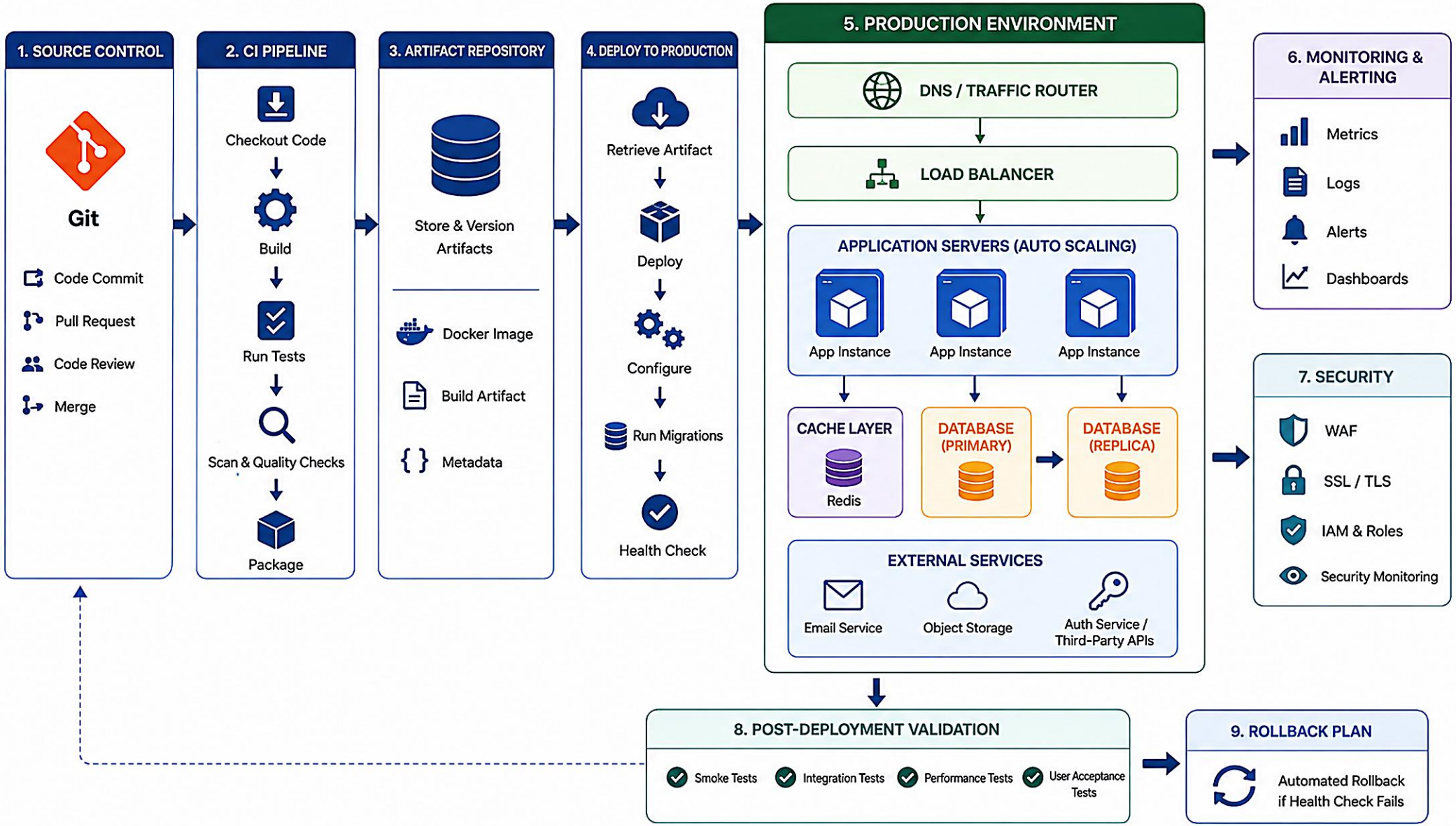
- Required approvals; environment protection rules; least-privilege secrets; automated health checks; audit logs and change history.

EXAMPLE: DEPLOY TO PRODUCTION

```
deploy-prod:
  needs: package
  if: startsWith(github.ref, 'refs/tags/')
  runs-on: ubuntu-latest
  environment:
    name: production
  steps:
    - uses: actions/download-artifact@v4
      with: { name: crm-jar }
    - name: Deploy to Production
      run: ./scripts/deploy.sh "$GITHUB_REF_NAME"
    - name: Health check
      run: ./scripts/health-check.sh prod
```

KEY TAKEAWAY Controlled production deployments ensure only high-quality, approved code reaches end users with minimal risk, maximum reliability, and full visibility.

Production Deployment



APPROVAL GATES

Approval gates are manual or policy-based checkpoints in the CI/CD pipeline that require explicit authorization before proceeding -- ensuring quality, security, and compliance.

KEY BENEFITS

- **Risk Mitigation --** block unverified or high-risk changes.
- **Quality Assurance --** ensure proper validation before promotion.
- **Compliance --** meet regulatory and audit requirements.
- **Visibility & Control --** track who approved, when, and why.

HOW APPROVAL GATES WORK IN GITHUB

#	Step
1	Configure required reviewers on the environment's protection rules.
2	The pipeline reaches that environment's job and pauses.
3	Reviewers are notified and review changes, tests, and artifacts.
4	A reviewer approves or rejects; the pipeline continues or stops.

TYPES OF APPROVAL GATES

Manual Approval **Timed Approval** **Policy-Based**

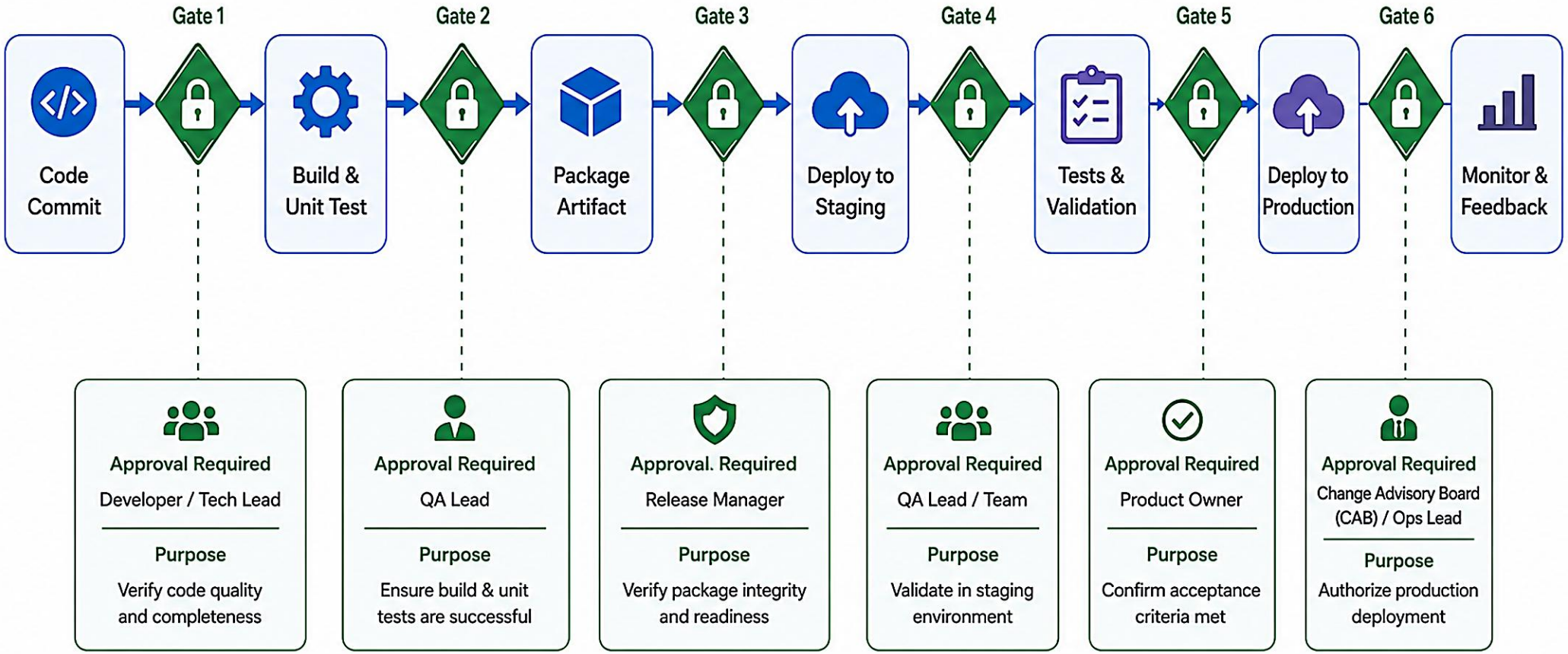
Multi-Layer

BEST PRACTICES

- Use least-privilege, required reviewers per environment.
- Keep approval groups small and accountable.
- Automate as much as possible before the gate; document policy.

KEY TAKEAWAY Approval gates add a human checkpoint where it matters most -- balancing speed with control so only validated, approved changes reach production.

Approval Gates



TRY IT YOURSELF -- EX 5: SECRETS CHECKLIST

Reinforces: approval gates protect WHO can deploy -- secrets discipline protects WHAT credentials the pipeline can see.

SORT EACH ITEM (notes/lab43-secrets-checklist.md)

Item	OK in Git?	Where It Belongs
Workflow YAML, README	Yes	Committed to the repository
Registry password, kubeconfig, .env	No	GitHub Actions secrets / variables
Scan reports (sanitized)	Yes	Committed or uploaded as artifacts

STEPS

- Step 1 -- Sort every item above; only non-secret config belongs in Git.
- Step 2 -- Write a three-step leak response: rotate the credential, purge history per policy, notify the instructor.
- Step 3 -- Note that CRM fixtures (CUS-1001, CUS-1002) are not secrets -- but real customer data always is.

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 5: exercises/exercise-05-secrets-checklist.md.

INTRODUCTION TO CODE INSIGHTS

Code Insights in GitHub provide automated analysis of your codebase to help you improve quality, security, maintainability, and performance throughout the CI/CD pipeline.

KEY CAPABILITIES

- **Code Quality --** detect bugs, code smells, and complexity issues.
- **Security Analysis --** find vulnerabilities and misconfigurations.
- **Maintainability --** measure complexity, duplication, and technical debt.
- **Trends Over Time --** track code health as your codebase evolves.

POPULAR ANALYSIS TOOLS

Tool	Focus
CodeQL	Security analysis, built into GitHub.
SonarQube	Code quality and technical debt.
Dependabot	Dependency vulnerability alerts.
Snyk	Security scanning for dependencies and code.

WHERE IT FITS IN THE PIPELINE

BEST PRACTICES

- Run Code Insights on every PR and the main branch.
- Fail builds on critical and high-severity issues.
- Define quality gates and review findings regularly.

KEY TAKEAWAY Code Insights brings automated intelligence into your pipeline -- fix issues early, keep your codebase healthy.

Introduction to Code Insights

AI-powered code analysis and intelligence for better software quality

WHAT IS CODE INSIGHTS?

Code Insights provides AI-driven analysis of your codebase to help developers write better code, fix issues faster, and maintain high-quality standards.

KEY CAPABILITIES

- Bug Detection
- Security Vulnerabilities
- Code Quality Metrics
- Code Smells & Anti-patterns
- Best Practices
- Actionable Insights

1. CONNECT → 2. ANALYZE → 3. INSIGHTS → 4. ACT → 5. IMPROVE

1. **CONNECT** — Connect your repositories
2. **ANALYZE** — AI analyzes code continuously
3. **INSIGHTS** — Get insights and recommendations
4. **ACT** — Take action to improve code
5. **IMPROVE** — Better code quality over time

HOW CODE INSIGHTS WORKS

SOURCE CODE	ANALYSIS ENGINE	KNOWLEDGE BASE	RESULTS	DASHBOARDS
Pull Requests Commits Repositories	AI/ML Models Static Analysis Data Flow Analysis	Rule Sets Security Databases Best Practices	Issues Vulnerabilities Recommendations	Reports Trends Metrics

SUPPORTED LANGUAGES

Java · Python · JavaScript · TypeScript · C# · C/C++ · Go · and more...

BENEFITS

- Improve code quality and reliability
- Identify and fix security vulnerabilities early
- Save time with actionable insights
- Enforce coding standards consistently
- Track progress and drive continuous improvement

INTEGRATIONS

- GitHub
- GitLab
- Bitbucket
- Azure DevOps
- Jenkins
- ...and more

TYPES OF INSIGHTS

BUGS
Detect potential bugs before they reach production

SECURITY
Find security flaws and protect your applications

CODE QUALITY
Improve maintainability, readability, and performance

CODE SMELLS
Identify anti-patterns and technical debt

BEST PRACTICES
Follow industry standards and coding best practices

CODE QUALITY METRICS

Code quality metrics quantify the health of your codebase, helping teams detect issues early, reduce technical debt, and deliver reliable software.

KEY METRICS

- **Bugs --** potential defects that can cause unexpected behavior.
- **Vulnerabilities --** security issues that could be exploited.
- **Code Smells --** maintainability issues that may cause future problems.
- **Coverage --** percentage of code covered by automated tests.
- **Duplications & Complexity --** duplicated code and how hard code is to understand.

QUALITY GATE EXAMPLE (THRESHOLDS)

Metric	Operator	Threshold	Status
Reliability Rating	worse than	A	Pass
Security Rating	worse than	A	Pass
Coverage	less than	70%	Pass
Duplications	greater than	10%	Fail
Code Smells	greater than	20	Pass

BEST PRACTICES

Define clear thresholds | **Use gates in every pipeline** | **Fail on critical/major issues** | **Track trends over time**

KEY TAKEAWAY Code quality metrics turn code into data -- measure, enforce, and improve to build better software, faster and safer.

Code Quality Metrics

1. Code Coverage
Measures the percentage of code covered by automated tests.

85%

Target: ≥ 80%

2. Defect Density
Number of confirmed defects per KLOC (thousand lines of code).

0.35

defects / KLOC

Target: ≤ 0.5

3. Security Vulnerabilities
Count of security issues identified by static analysis tools.

5

Open Issues

Target: 0

4. Maintainability Index
Indicates how easy the code is to maintain (higher is better).

78

out of 100

Target: ≥ 70

5. Code Smells
Potential maintainability issues identified by static analysis.

23

Code Smells

Target: < 20

6. Technical Debt
Estimated effort (in hours) required to fix all code issues.

42h

Total Debt

Target: Decreasing

WHAT DO THESE METRICS TELL US?

- Track overall code health
- Detect issues early
- Measure progress over time
- Improve quality and productivity
- Reduce risks and technical debt

QUALITY SCORE

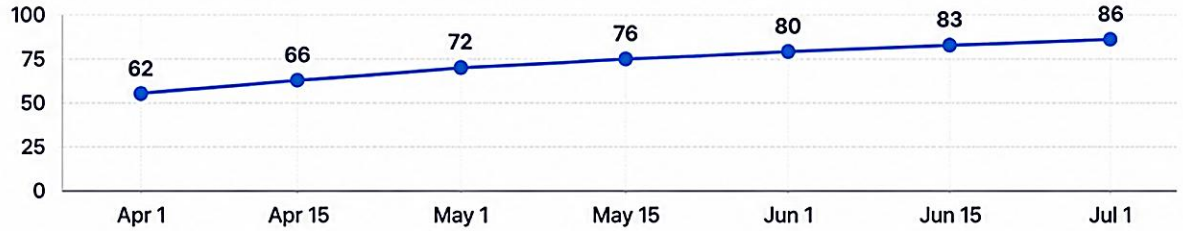
86
/100

Overall Quality Score

Aggregated score based on all quality metrics.

Good

QUALITY TREND OVER TIME



Quality is improving!

PIPELINE BEST PRACTICES

Well-designed pipelines are reliable, secure, fast, and maintainable. Following best practices ensures consistent delivery, reduces risk, and improves developer productivity.

CORE PRINCIPLES

Automate Everything | **Fail Fast** | **Keep It Fast & Simple** | **Secure by Default** | **Ensure Visibility** | **Continuously Improve**

PIPELINE BEST PRACTICES CHECKLIST

Practice	What To Do
Version Everything	Version source code, pipeline configs, and dependencies; use immutable tool/action versions.
Small, Focused Pipelines	Break pipelines into reusable jobs; one responsibility per job.
Use Caching Wisely	Cache dependencies and build outputs to speed up every run.
Enforce Quality Gates	Define thresholds for quality, security, and tests; fail the pipeline if not met.
Use Immutable Artifacts	Build once, promote the same artifact -- never rebuild between environments.
Limit Permissions	Restrict token scopes and environment access to only what's necessary.

PRO TIP

Start simple, then iterate. Measure and optimize continuously. Treat pipelines as production code.

KEY TAKEAWAY A great pipeline is built on automation, security, quality, and continuous improvement -- follow best practices to deliver value faster and with confidence.

Pipeline Best Practices

1 KEEP IT SIMPLE AND AUTOMATED

- Automate everything possible
- Avoid manual interventions
- Keep the pipeline easy to understand and maintain

2 BUILD EARLY, TEST OFTEN

- Build frequently
- Run automated tests at every stage
- Catch issues early

3 KEEP PIPELINES FAST

- Optimize build and test time
- Run tasks in parallel
- Use caching and incremental builds

4 USE VERSION CONTROL

- Store pipeline code in Git
- Version your pipeline configurations
- Review changes via pull requests

5 ENABLE QUALITY GATES

- Define quality criteria
- Enforce gates for tests, security scans, and code quality
- Block bad builds from progressing

6 SECURE YOUR PIPELINES

- Manage secrets securely
- Use least-privilege access
- Scan dependencies and images for vulnerabilities

7 MAKE IT REPRODUCIBLE

- Use immutable environments
- Pin versions and dependencies
- Ensure consistent results across runs

8 MONITOR AND OBSERVE

- Monitor pipeline health
- Collect and visualize metrics
- Set up alerts for failures and bottlenecks

9 HANDLE FAILURES EFFECTIVELY

- Fail fast and provide clear logs
- Notify the right people
- Allow easy re-runs and rollback strategies

10 CONTINUOUSLY IMPROVE

- Review pipeline performance
- Remove bottlenecks
- Iterate and improve continuously

FOUNDATIONAL PRINCIPLES

Reliability
Build trust in your pipeline

Collaboration
Enable visibility and team collaboration

Consistency
Standardize processes and environments

Efficiency
Optimize for speed and cost

Compliance
Meet security and regulatory standards

TRY IT YOURSELF -- EX 6: CI RUNBOOK (CAPSTONE)

Capstone: outline docs/ci-runbook.md so a peer can re-run a failed verify -- then confirm every pre-lab exercise is Pass.

RUNBOOK OUTLINE HEADINGS (notes/lab43-ci-runbook-outline.md)

Heading	Contents
Policy	Triggers, jobs, where reports live.
Re-run	GitHub UI/CLI re-run path + local ./mvnw -B clean verify equivalent.
Gates	Surefire/Failsafe must be green; documented scan threshold.
Secrets	Names only -- never values -- of secured variables.

PRE-LAB READINESS -- DONE WHEN

- All six notes/lab43-*.md files exist (pipeline policy, verify plan, immutable jar, workflow TODOs, secrets checklist, runbook outline).
- Fixtures match Amina CUS-1001/ACTIVE and Ravi CUS-1002/PROSPECT wherever used.
- Self-mark every exercise Pass, then open the Lab 43 OS-specific guide (Windows or macOS).

KEY TAKEAWAY TRY IT NOW -- Capstone: exercises/exercise-06-ci-runbook-outline.md, then confirm all six exercise notes files are Pass before starting Lab 43.

LAB OVERVIEW -- CI/CD PIPELINE FOR THE CRM

Lab 43: GitHub CI/CD Pipeline for the CRM -- Northstar Delivery Gates.

BUSINESS SCENARIO

- Leadership freezes a delivery rule: pull requests get fast feedback; main and version tags get stronger gates; deployment credentials never live in Git; the JAR verified in CI is the JAR promoted later -- not a silently rebuilt binary.
- A green demo without evidence is not enough -- you own the CI contract that verifies the CRM backend serving Amina (CUS-1001) and Ravi (CUS-1002).
- Lab 44 promotes the same immutable artifact identity you freeze here.

LEARNING OBJECTIVES

- Model CI stages and gates for PR, main, and version tags.
- Configure Maven dependency caching in GitHub Actions.
- Publish Surefire, Failsafe, and security-scan reports as artifacts.
- Pass immutable artifacts (JAR + checksum + commit) between isolated steps.
- Protect deployment variables as secured, scoped secrets.
- Force a safe pipeline failure, interpret evidence, and document rerun policy.

WHAT YOU BUILD

- lab43-crm's .github/workflows/ci.yml (verify + package-once jobs), Surefire/Failsafe + scan report evidence, a checksummed JAR tied to GITHUB_SHA, docs/ci-runbook.md, and a controlled failure-then-restore evidence pack.

KEY TAKEAWAY Fixtures CUS-1001 (Amina Khan, ACTIVE) and CUS-1002 (Ravi Singh, PROSPECT) anchor smoke evidence; correlation id lab-request-001 labels pipeline evidence; GITHUB_SHA ties the checksummed JAR to its commit.

LAB TASKS AND DELIVERABLES

GitHub CI/CD Pipeline for the CRM -- implementation steps, deliverables, and the evaluation rubric.

#	Implementation Step
1	Define pipeline policy for PR / main / tags in docs/ci-runbook.md.
2	Prepare reproducible commands: JDK 21, Maven Wrapper, clean verify (no skipped tests).
3	Author ci.yml verify job with Maven cache and Surefire/Failsafe report artifacts.
4	Add a dependency-scan / SAST quality gate with a documented fail threshold.
5	Add a package job (needs: verify) that writes SHA-256 + commit + build number.
6	Configure branch behavior: focused PR checks, complete main gates, manual tag deploy.
7	Protect deployment variables as secured, environment-scoped GitHub secrets.
8	Push, run the pipeline, force one safe failure, restore green, document the runbook.
9	Complete failure experiments; capture sanitized evidence; confirm no secrets in Git.

EXPECTED DELIVERABLES

- .github/workflows/ci.yml with PR/main/tag behavior.
- Test report evidence (Surefire/Failsafe).
- Security/scan report evidence or documented residual risk.
- JAR and SHA-256 checksum tied to the commit.
- docs/ci-runbook.md with policy, rerun, and troubleshooting.
- Controlled failure-path result, then restore.

RUBRIC (100 MARKS)

- Environment 10 · Core Impl 30 · Integration/Config 15 · Failure Handling 15 · Verification 10 · Security/Prod Awareness 10 · Docs 10

KEY TAKEAWAY Green pipeline that skips tests or rebuilds on deploy loses security/operability marks; committed secrets are an honor violation until remediated.

MODULE 43 SUMMARY

In this module, we learned how to design and implement a complete CI/CD pipeline using GitHub Actions -- build, test, scan, package, and deploy with confidence.

KEY CONCEPTS COVERED

CI/CD Fundamentals

The CI/CD lifecycle, key concepts, and how automation improves quality and speed.

GitHub Actions & Pipelines

Workflows, events, runners, jobs, steps, and YAML configuration.

Quality & Security Gates

Automated tests, security scans, Code Insights, and quality metrics.

Artifact Management

Package-once, versioned, checksummed artifacts promoted -- never rebuilt.

Deployment Automation

Staging and production deploys with environments, approval gates, and rollback.

Best Practices

Secure, efficient, maintainable pipelines applied to the real CRM lab.

MODULE FLOW RECAP

1. **CI/CD Foundations**
2. **GitHub Actions**
3. **Build/Test/Scan/Package**
4. **Deploy with Gates**
5. **Lab: CRM Pipeline**

KEY TAKEAWAY GitHub Actions enables end-to-end automation of the software delivery lifecycle -- from commit to production -- with quality gates, secrets protection, and full traceability.

KNOWLEDGE CHECK (1 OF 2)

Test your understanding of GitHub Actions and CI/CD pipelines -- questions 1-4 of 8.

1. What is the primary purpose of GitHub Actions?

A. To store source code

B. To automate the build, test, and deployment process

C. To manage infrastructure

D. To design UI/UX

2. Which filename is used for the CRM's CI/CD workflow in this course?

A. actions.yml

B. pipeline.yaml

C. ci.yml

D. workflow.json

3. Where should GitHub Actions workflow files be placed in a repository?

A. /config

B. .github/workflows

C. /pipeline

D. /actions

4. Which event triggers a workflow to run when code is pushed to the repository?

A. schedule

B. push

C. workflow_dispatch

D. issue_comment

KEY TAKEAWAY GitHub Actions automates build/test/deploy; workflow files live in .github/workflows; push triggers a workflow on new commits.

KNOWLEDGE CHECK (2 OF 2)

Four more questions on testing, tooling, artifacts, and approval gates -- questions 5-8 of 8.

5. What is the main benefit of using automated tests in the pipeline?

A. Increase manual work

B. Catch defects early and improve quality

C. Slow down delivery

D. Replace code reviews

6. Which tool is commonly used for static code analysis alongside GitHub Actions?

A. SonarQube

B. Postman

C. Selenium

D. JMeter

7. What does an "artifact" in a pipeline usually refer to?

A. Source code repository

B. Build output like a JAR, reports, or packages

C. Production server

D. User interface

8. Why are approval gates important before production deployment?

A. To add more steps to the pipeline

B. To control and govern releases

C. To run more tests

D. To restart the pipeline

KEY TAKEAWAY Automated tests catch defects early; SonarQube analyzes code statically; an artifact is a build output like a JAR; approval gates control and govern releases.

"Automate. Secure. Deliver. Repeat."

MODULE 43 COMPLETE

GitHub Actions and CI/CD Integration

You can now design and implement a reliable, secure, automated GitHub Actions CI/CD pipeline -- from commit to production -- for the Northstar CRM.